Cheatsheet: Parsers Funcionales

> Pseudo-código estilo Haskell (no compila tal cual). La idea es recordar **qué hace** cada función.

El tipo

```
Parser a = String -> [(a, String)]
-- toma una cadena y devuelve:
--   []            -> falló
--   [(v, resto)]  -> éxito: valor v y lo que queda sin consumir

parse :: Parser a -> String -> [(a, String)]
parse p inp = p inp
-- aplica el parser p a la cadena inp
```

Primitivas

```
item :: Parser Char
-- consume UN caracter; falla si la entrada está vacía
-- parse item "abc"  =>  [('a', "bc")]
-- parse item ""     =>  []

failure :: Parser a
-- siempre falla
-- parse failure "abc"  =>  []

return :: a -> Parser a
return v
-- siempre tiene éxito, devuelve v SIN consumir nada
-- parse (return 1) "abc"  =>  [(1, "abc")]

(<|>) :: Parser a -> Parser a -> Parser a
p <|> q
-- elección: prueba p; si falla, prueba q con la MISMA entrada
-- si p tiene éxito, q ni se mira (no hay vuelta atrás)
-- parse (item <|> return 'd') "abc"     =>  [('a', "bc")]
-- parse (failure <|> return 'd') "abc"  =>  [('d', "abc")]
```

Secuenciación (do)

```
p :: Parser (Char, Char)
p = do x <- item      -- guardo el 1er caracter en x
       item           -- consumo el 2do y lo descarto
       y <- item      -- guardo el 3ro en y
       return (x, y)  -- el resultado de todo el bloque

-- parse p "abcdef"  =>  [(('a','c'), "def")]
-- parse p "ab"      =>  []    -- si un paso falla, falla todo
```

- `x <- p` → corre `p` y le pone nombre al resultado
- `p` solo → corre `p` y descarta el resultado
- el último parser da el valor final
- todas las líneas alineadas en la misma columna

Derivadas

```
sat :: (Char -> Bool) -> Parser Char
-- consume un caracter SOLO si cumple el predicado
sat pred = do x <- item
              if pred x then return x else failure

digit :: Parser Char
digit = sat isDigit     -- un dígito '0'..'9'

char :: Char -> Parser Char
char c = sat (== c)     -- exactamente el caracter c

many :: Parser a -> Parser [a]
-- aplica p CERO o más veces (nunca falla, puede dar [])
many p = many1 p <|> return []
-- parse (many digit) "123abc"   =>  [("123", "abc")]
-- parse (many digit) "abc"      =>  [("", "abc")]      -- cero veces: éxito igual
-- parse (many (char 'a')) "aab" =>  [("aa", "b")]
-- parse (many item) "hola"      =>  [("hola", "")]     -- consume todo

many1 :: Parser a -> Parser [a]
-- aplica p UNA o más veces (falla si no hay al menos una)
many1 p = do v  <- p
             vs <- many p
             return (v : vs)
-- parse (many1 digit) "123abc"  =>  [("123", "abc")]
-- parse (many1 digit) "abc"     =>  []                 -- ninguna vez: falla
-- parse (many1 (char 'a')) "a"  =>  [("a", "")]

-- uso típico: un número de varios dígitos
numero :: Parser Int
numero = do ds <- many1 digit
            return (read ds)
-- parse numero "42+1"  =>  [(42, "+1")]
-- parse numero "+1"    =>  []

-- uso típico: saltear espacios (puede no haber ninguno)
espacios :: Parser ()
espacios = do many (char ' ')
              return ()
-- parse espacios "   x"  =>  [((), "x")]
-- parse espacios "x"     =>  [((), "x")]

string :: String -> Parser String
-- reconoce exactamente la cadena dada
string []     = return []
string (x:xs) = do char x
                   string xs
                   return (x:xs)
```

Ejemplo: lista [1,2,3,4]

```
lista :: Parser String
lista = do char '['
           d  <- digit
           ds <- many (do char ','
                          digit)
           char ']'
           return (d : ds)

-- parse lista "[1,2,3,4]"  =>  [("1234", "")]
-- parse lista "[1,2,3,4"   =>  []
```

Expresiones aritméticas

Gramática (asocia a derecha, * antes que +)

```
expr   -> term '+' expr | term
term   -> factor '*' term | factor
factor -> digit | '(' expr ')'
```

Factorizada (no repite el prefijo común)

```
expr -> term ('+' expr | ε)
term -> factor ('*' term | ε)
```

¿Qué significa factorizar?

Es "sacar factor común", igual que en álgebra:

```
a·b + a·c   =   a·(b + c)
```

Si dos alternativas **empiezan igual**, ese prefijo se saca afuera:

```
expr -> term '+' expr | term          -- las dos empiezan con term
         ↓ sacamos term
expr -> term ('+' expr | ε)           -- ε = cadena vacía ("o nada")
```

En general: `A -> α β | α γ` pasa a ser `A -> α (β | γ)`.

Reconoce exactamente las mismas cadenas. Solo cambia la forma de escribirla y, por lo tanto, lo que hace el parser.

¿Por qué hay que factorizar?

`<|>` prueba la primera alternativa y, si falla, **vuelve al principio** y prueba la segunda con la misma entrada. Todo lo que avanzó la primera se pierde.

Sin factorizar, con la entrada `"2*3"`:

```
expr -> term '+' expr | term

1. Alternativa 1: parsea term  -> lee "2*3"
                  espera '+'   -> no hay, FALLA
2. Alternativa 2: parsea term  -> lee "2*3" OTRA VEZ desde cero
```

Factorizada, con la entrada `"2*3"`:

```
expr -> term ('+' expr | ε)

1. parsea term  -> lee "2*3"   (una sola vez)
2. ¿hay '+'? no -> ε, devuelvo el valor de term
```

El costo crece exponencialmente

Sin factorizar, cada nivel **multiplica** el trabajo:

- `expr` puede llamar a `term` **2 veces** (si no hay `+`)
- cada `term` puede llamar a `factor` **2 veces** (si no hay `*`)
- cada `factor` con paréntesis vuelve a llamar a `expr`

Cada nivel de paréntesis multiplica el trabajo por 2 × 2 = **4**:

Entrada	Llamadas aprox. sin factorizar	Factorizada
`1`	4	1
`(1)`	16	2
`((1))`	64	3
`((((((((((1))))))))))` (10 niveles)	$4^{10} \approx$ **1.000.000**	11

Sin factorizar el costo es **exponencial** (4^n). Factorizada es **lineal**: cada parte de la entrada se lee una sola vez.

Parser = traducir cada regla

```
expr :: Parser Int
expr = do t <- term
          (do char '+'          -- si viene un '+'...
              e <- expr
              return (t + e))
          <|> return t          -- ...si no, ε: devuelvo t

term :: Parser Int
term = do f <- factor
          (do char '*'
              t <- term
              return (f * t))
          <|> return f

factor :: Parser Int
factor = (do d <- digit
             return (digitToInt d))
     <|> (do char '('
             e <- expr
             char ')'
             return e)

eval :: String -> Int
eval xs = fst (head (parse expr xs))
-- eval "2*3+4"    =>  10
-- eval "2*(3+4)"  =>  14
-- OJO: explota si no parsea y no controla que se consuma toda la entrada
```

`eval` seguro

```
eval :: String -> Int
eval xs = case parse expr xs of
  [(n, "")]  -> n                      -- todo consumido: ok
  [(_, out)] -> error "sobró entrada"
  []         -> error "entrada inválida"
```

Problemas y soluciones

Problema	Causa	Solución
Parser lento	`term '+' expr | term` parsea `term` dos veces	**Factorizar**: `term ('+' expr | ε)`
`8-4-2 = 6` (mal)	La gramática asocia a derecha: `8-(4-2)`	Gramática recursiva a izquierda...
Bucle infinito	`expr -> expr '+' term`: `expr` se llama sin consumir nada (**recursión a izquierda**)	Transformar la gramática

¿Por qué la recursión a izquierda genera un bucle infinito?

Un parser es una **función** que recibe la entrada. Con recursión a izquierda, lo **primero** que hace `expr` es llamarse a sí mismo **con la misma entrada** (todavía no consumió nada). Esa llamada hace exactamente lo mismo, y así sin fin.

```
expr :: Parser Int
expr = do t <- expr        -- ← lo primero: llamar a expr
          char '+'
          e <- term
          return (t + e)
   <|> term
```

Paso a paso con `"1+2"`:

```
expr "1+2"
 └─ alternativa 1: primero llama a expr "1+2"
     └─ alternativa 1: primero llama a expr "1+2"
         └─ alternativa 1: primero llama a expr "1+2"
             └─ ...  (para siempre)
```

- La entrada **nunca se achica**: siempre es `"1+2"`.
- Nunca se llega a `char '+'`: antes hay que terminar la llamada a `expr`, que nunca termina.
- Tampoco se llega a `<|> term`: `<|>` solo prueba la segunda alternativa **cuando la primera falla**, y la primera ni falla ni tiene éxito, nunca termina.

En la práctica: el programa se cuelga o da *stack overflow*.

Por qué la recursión a derecha sí funciona

```
expr = do t <- term        -- ← primero consume algo
          char '+'         -- ← y esto también
          e <- expr        -- recién acá se llama a sí mismo
          return (t + e)
   <|> term
```

```
expr "1+2+3"
 └─ term consume "1", char consume "+"
     └─ expr "2+3"                      ← entrada más corta
         └─ term consume "2", char consume "+"
             └─ expr "3"                ← más corta todavía
                 └─ term consume "3", no hay '+' → termina
```

Cada llamada recursiva recibe una **entrada más corta**. Como la entrada es finita, la recursión termina.

La regla

	Antes de la llamada recursiva	Entrada en la llamada	Resultado
`A → A α` (izquierda)	nada	**la misma**	bucle infinito
`A → β A` (derecha)	se consume `β`	**más corta**	termina

> Una llamada recursiva es segura solo si **antes se consumió al menos un caracter**.

Por eso la transformación de abajo funciona:

- `A` empieza por `β`, que no empieza con `A`.
- Cada vuelta de `A'` consume `α` antes de volver a llamarse. Por eso `α` tiene que ser no vacío.

Eliminar recursión a izquierda

```
A  -> A α | β          (α no vacío, β no empieza con A)

se transforma en:

A  -> β A'
A' -> ε | α A'
```

Ejemplo:

```
resto  -> resto 'sigue' resto | 'valor'
```

Identificamos las partes comparando con `A -> A α | β`:

Parte	Valor en el ejemplo
`A`	`resto`
`α`	`'sigue' resto` (lo que sigue a `resto` en la alternativa recursiva)
`β`	`'valor'` (la alternativa que NO empieza con `resto`)

Reemplazando en `A -> β A'` y `A' -> ε | α A'`:

```
resto  -> 'valor' resto'
resto' -> ε | 'sigue' resto resto'
```

Mismo procedimiento para `expr -> expr '+' term | term`:

Parte	Valor
`A`	`expr`
`α`	`'+' term`
`β`	`term`

```
expr  -> term expr'
expr' -> ε | '+' term expr'
```

Aplicado a expresiones (con - y /)

```
expr  -> term expr'
expr' -> '+' term expr' | '-' term expr' | ε
term  -> factor term'
term' -> '*' factor term' | '/' factor term' | ε
```

```
-- truco: expr' recibe lo acumulado hasta ahora => asocia a IZQUIERDA
expr :: Parser Int
expr = do t <- term
          expr' t

expr' :: Int -> Parser Int     -- recibe el acumulado
expr' acc = (do char '+'
                t <- term
                expr' (acc + t))
        <|> (do char '-'
                t <- term
                expr' (acc - t))
        <|> return acc            -- ε: no hay más operadores

term :: Parser Int
term = do f <- factor
          term' f

term' :: Int -> Parser Int     -- recibe el acumulado
term' acc = (do char '*'
                f <- factor
                term' (acc * f))
        <|> (do char '/'
                f <- factor
                term' (acc / f))
        <|> return acc

-- eval "8-4-2"  =>  2   ((8-4)-2)
-- eval "8/4/2"  =>  1   ((8/4)/2)
```

Resumen en 3 líneas

1. Un parser reconoce la estructura sintáctica de una cadena.
2. Parsers complejos = combinar (`<|>`) y secuenciar (`do`) parsers básicos.
3. Hay que **factorizar** (eficiencia) y **eliminar recursión a izquierda** (si no, bucle infinito).